Lab 7.1: Using CAccessor

In this lab, you will create a consumer application by using the **CAccessor** class. You will add, delete, and update records in a database table.

To see a demonstration of the solution to this lab, click this icon.
(CD-ROM plays the demonstration, "Lab 7.1 Demonstration.")

Estimated time to complete this lab: **30 minutes**

To complete the exercises in this lab, you must have the required software. For detailed information about the labs and setup for the labs, see Labs in this course.

The solution code for this lab is located in the folder \Labs\Ch07\Lab7.1\Ex01\Solution.

Objectives

After completing this lab, you will be able to:

- Create a consumer application by using the **CAccessor** class.
- Add, update, and delete records in a database table.

Prerequisites

There are no prerequisites for this lab.

Exercises

The following exercise provides practice working with the concepts and techniques covered in this chapter.

Exercise 1: Performing Database Operations

In this exercise, you will implement add, delete, and update functions to manipulate records in a database table.

230 Exercise 1: Performing Database Operations

In this exercise, you will implement add, delete, and update functions to manipulate records in a database table.

▶ **Create an application by using MFC AppWizard**

☐ 1. Copy the Employees database from \Labs\Ch07\Lab7.1\Ex01 into your project folder.

☐ 2. Invoke MFC AppWizard.

☐ 3. Name the new project workspace **OledbApp**.

☐ 4. In Step 1 of MFC AppWizard, click **single document**, and then click **Next**.

☐ 5. In Step 2 of AppWizard, click **Database view with file support** and then click **Data Source**.

The **Database Options** dialog box appears.

☐ 6. Select **OLE DB** and then click **Select OLE DB Datasource**.

The **Data Link Properties** dialog box appears.

☐ 7. Select **Microsoft Jet 3.51 OLE DB Provider** from the list of providers and click **Next**.

☐ 8. Enter the name of the database file.

☐ 9. Enter the user name and password to connect to the data source.

☐ 10. Click **Test Connection** to test the connection to the provider and click **OK**.

☐ 11. Click **OK** to close the **Database Options** dialog box.

☐ 12. Select the **Employees** table and click **OK**.

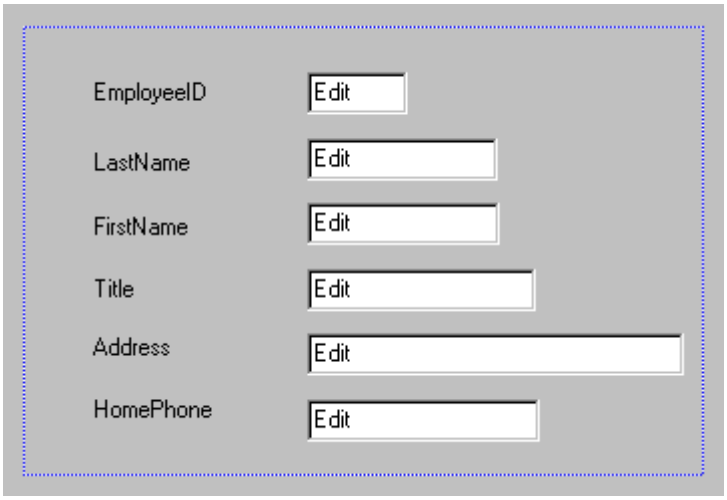
☐ 13. In Step 4 of MFC AppWizard, clear the **Printing and print preview** option.

☐ 14. Click **Finish** to create the skeletal program for the application.

Notes

▶ Add controls to IDD_OLEDBAPP_FORM dialog box

In this procedure, you will create a form as shown in the following illustration.



- In IDD_OLEDBAPP_FORM, add the following controls to the form.

Control type	ID	Caption
Static text	**IDC_STATIC**	EmployeeID
Edit box	**ID_EMPID**	None
Static text	**IDC_STATIC**	LastName
Edit box	**ID_LNAME**	None
Static text	**IDC_STATIC**	FirstName
Edit box	**ID_FNAME**	None
Static text	**IDC_STATIC**	Title
Edit box	**ID_TITLE**	None
Static text	**IDC_STATIC**	Address
Edit box	**ID_ADDRESS**	None
Static text	**IDC_STATIC**	HomePhone
Edit box	**ID_PHONE**	None

Notes

▶ Add DDX functions

In this procedure, you will add DDX functions to handle the transfer of data between the controls and the recordset variables.

- In the **DoDataExchange** function of the OledbAppView.cpp file, add DDX functions after the //{{AFX_DATA_MAP block as follows:

```
//{{AFX_DATA_MAP(COledbAppView)
//}}AFX_DATA_MAP
DDX_Text(pDX, ID_EMPID, m_pSet->m_EmployeeID);
DDX_Text(pDX, ID_LNAME, m_pSet->m_LastName,21);
DDX_Text(pDX, ID_FNAME, m_pSet->m_FirstName,11);
DDX_Text(pDX, ID_TITLE, m_pSet->m_Title,31);
DDX_Text(pDX, ID_ADDRESS, m_pSet->m_Address,61);
DDX_Text(pDX, ID_PHONE, m_pSet->m_HomePhone,25);
```

▶ Modify the IDR_MAINFRAME menu resource

☐ 1. Open the IDR_MAINFRAME menu resource.

☐ 2. Delete the **Edit** menu and all menu items under **Edit**.

☐ 3. Add the following menu items to the **Record** menu.

ID	Caption
ID_RECORD_CLEAR	Clear
ID_RECORD_ADD	Add Record
ID_RECORD_DELETE	Delete Record
ID_RECORD_COMMIT	Commit

▶ Modify the IDR_MAINFRAME toolbar resource

☐ 1. Open the IDR_MAINFRAME toolbar resource.

☐ 2. Add four toolbar items, giving them the IDs ID_RECORD_CLEAR, ID_RECORD_ADD, ID_RECORD_DELETE, and ID_RECORD_COMMIT, respectively.

Your toolbar should look like the following toolbar.

▶ Create message map entries

In this procedure, you will create message map entries for the menu items on the **Record** menu.

☐ 1. Open COledbAppView.h and add the following entries after the //{{AFX_MSG message:

```
afx_msg void OnRecordPrev();
afx_msg void OnRecordNext();
afx_msg void OnRecordClear();
afx_msg void OnRecordAdd();
afx_msg void OnRecordDelete();
afx_msg void OnRecordCommit();
```

Notes

☐ 2. Insert message map entries for ON_COMMAND in the **COledbAppView** implementation file after the //{{AFX_MSG message as follows:

```
ON_COMMAND(ID_RECORD_PREV, OnRecordPrev)
ON_COMMAND(ID_RECORD_NEXT, OnRecordNext)
ON_COMMAND(ID_RECORD_CLEAR, OnRecordClear)
ON_COMMAND(ID_RECORD_ADD, OnRecordAdd)
ON_COMMAND(ID_RECORD_DELETE, OnRecordDelete)
ON_COMMAND(ID_RECORD_COMMIT, OnRecordCommit)
```

▶ **Implement the OnRecordPrev and OnRecordNext handlers**

☐ 1. In the OledbAppView.cpp file, write the code for the **OnRecordPrev** handler as follows:

```
void COledbAppView::OnRecordPrev()
{
    if(!OnMove(ID_RECORD_PREV))
    OnMove(ID_RECORD_FIRST);
}
```

☐ 2. In the OledbAppView.cpp file, write the code for the **OnRecordNext** handler as follows:

```
void COledbAppView::OnRecordNext()
{
    if(!OnMove(ID_RECORD_NEXT))
    OnMove(ID_RECORD_LAST);
}
```

▶ **Implement the Clear function**

In this procedure, you will clear the values of the current record displayed on the form.

- In the OledbAppView.cpp file, write the code for the **OnRecordClear** handler as follows:

```
void COledbAppView::OnRecordClear()
{
    m_pSet->m_EmployeeID = 0;
    m_pSet->m_LastName[0] = '\0';
    m_pSet->m_FirstName[0]= '\0';
    m_pSet->m_Title[0]= '\0';
    m_pSet->m_Address[0]= '\0';
    m_pSet->m_HomePhone[0]= '\0';
    UpdateData(FALSE);
}
```

▶ **Implement the Add and Delete functions**

☐ 1. Open OledbAppSet.h.

☐ 2. In the **AddProperty** function of **propset**, change the second argument as follows to allow addition and deletion of records:

```
propset.AddProperty(DBPROP_UPDATABILITY,
DBPROPVAL_UP_CHANGE|DBPROPVAL_UP_INSERT|DBPROPVAL_UP_DELETE);
```

Notes

☐ 3. In the OledbAppView.cpp file, write the code for the **OnRecordAdd** handler as follows:

```
void COledbAppView::OnRecordAdd()
{
    UpdateData(TRUE);
    m_pSet->Insert();
    m_pSet->MoveLast();
    if(m_bOnFirstRecord && m_bOnLastRecord)
    {
        m_bOnFirstRecord = FALSE;
        m_bOnLastRecord = FALSE;

    }
}
```

☐ 4. Write the code for the **OnRecordDelete** handler as follows:

```
void COledbAppView::OnRecordDelete()
{
    m_pSet->Delete();
    if(!OnMove(ID_RECORD_NEXT))
    {
        if(!OnMove(ID_RECORD_LAST))
        {
            OnRecordClear();
            m_bOnFirstRecord = TRUE;
            m_bOnLastRecord = TRUE;
        }
    }
    UpdateData(FALSE);
}
```

▶ **Implement the Commit function**

In this procedure, you will use the **Commit** member of the session object to write the changes in the recordset to the data source.

☐ 1. Delete the local instance of the **CSession** object from the **Open** function of the **COledbAppSet** class.

☐ 2. Declare a public instance of the **CSession** object in the **COledbAppSet** class to make the session object available outside the class:

```
CSession session;
```

☐ 3. In the **Open** function of OledbAppSet.h file, begin a transaction by invoking **StartTransaction** after the call to the CSession **Open** member function as follows:

```
session.StartTransaction();
```

☐ 4. In the OledbAppView.cpp file, write the code for the **OnRecordCommit** handler as follows:

```
void COledbAppView::OnRecordCommit()
{
    m_pSet->session.Commit();
}
```

Notes

▶ **Add an update handler for the Delete menu command**

☐ 1. Using ClassWizard, add an update handler for the **Delete** menu command.

☐ 2. Write code for the update handler as follows:

```
void COledbAppView::OnUpdateRecordDelete(CCmdUI* pCmdUI)
{
    if(pCmdUI->m_nID == ID_RECORD_DELETE)
    {
        if(m_bOnFirstRecord && m_bOnLastRecord)
            pCmdUI->Enable(FALSE);
        else
            pCmdUI->Enable(TRUE);
    }

}
```

The previous example code disables the **Delete** command if all the records in the table have been deleted.

▶ **Build and test your application**

- Run your application and test the options under the **Record** menu.

The solution code for this exercise is located in the folder \Labs\Ch07\Lab7.1\Ex01\Solution.

Notes